

04-代码生成与优化复习讲义

目录

1	04 代码生成与优化复习讲义
---	----------------

2

1 04 代码生成与优化复习讲义

对应资料：

- Slides/Slides/Ch8-1-Target Code Generation.pdf
- Slides/Slides/Ch8-2-Translation out of SSA.pdf
- Slides/Slides/Ch8-3-Register Allocation.pdf
- Slides/Slides/Ch10-Instruction Scheduling.pdf
- Slides/Slides/handout/4 Backend.pdf
- Review/compile.pdf
- Exam/2025.pdf 指令调度题
- Exam/过时/2023-exam-A.pdf 题目 6

1.1 一、考试目标

后端题重点：

目标代码生成：知道任务

SSA 转出：知道 `phi` 怎么拆

寄存器分配：知道冲突图和 `spill`

指令调度：会写三类依赖、画依赖图、给调度序列

2025 真题重点在指令调度。

1.2 二、目标代码生成

目标代码生成把 IR 变成机器代码。核心任务：

- 指令选择。
- 寄存器分配。
- 指令调度。
- 栈帧和调用约定。
- 地址计算。

答定义题写法:

```
Target code generation translates IR into target-machine instructions.
It must choose instructions, allocate registers, schedule instructions,
and preserve the program semantics.
```

1.3 三、寻址模式

常见写法:

LD R1, a(R2)	读内存地址 R2+a
LD R1, #100	立即数 100 放入 R1
LD R1, *R2	读 R2 指向地址的内容
ST a(R2), R1	把 R1 写到内存地址 R2+a

看到地址表达式:

```
base + offset
```

就写成:

```
offset(base)
```

1.4 四、局部优化

基本块内常见优化:

```
t1 = a + b
t2 = a + b
```

改成:

```
t1 = a + b
t2 = t1
```

常见名称:

- 公共子表达式消除。
- 常量折叠。
- 强度削弱。

- 冗余 load/store 删除。
- 窥孔优化。

例：

```
x * 2      -> x + x
x = x + 1   -> INC x
```

1.5 五、SSA 转出

phi:

```
x3 = phi(x1 from B1, x2 from B2)
```

转出方式：

```
B1 末尾插入 x3 = x1
B2 末尾插入 x3 = x2
删除 phi
```

多条 phi 要按并行复制处理：

```
x = y
y = x
```

不能顺序直接写。用临时变量：

```
t = x
x = y
y = t
```

答题模板：

1. 对每个 **phi** 输入，在对应前驱块末尾插入复制。
2. 同一前驱块的多条复制按并行赋值处理。
3. 有循环复制时引入临时变量。
4. 删除 **phi** 指令。

1.6 六、寄存器分配

目标：

无限虚拟寄存器 -> 有限物理寄存器

核心问题：

同一时刻都活跃的两个变量不能放在同一个物理寄存器。

冲突图：

顶点：变量或虚拟寄存器

边：两个变量活跃区间相交

k-染色：

能用 k 种颜色给冲突图染色，就能用 k 个物理寄存器分配。

染不了就 **spill**，把变量放到内存。

Chaitin 简化流程：

1. 找度数小于 k 的点，删掉并压栈。
2. 重复直到图空。
3. 找不到度数小于 k 的点时，选点 **spill**。
4. 出栈时给颜色。

局部寄存器分配经验：

寄存器不够时，**spill** 下一次使用距离最远的变量。

1.7 七、指令调度

目的：

重排不改变语义的指令，减少流水线停顿，提高并行执行效率。

三类依赖必须会写：

True dependence / RAW:

i 写 x , j 读 x , i 必须在 j 前。

Anti dependence / WAR:

i 读 x , j 写 x , j 不能提前破坏 i 读到的旧值。

Output dependence / WAW:

i 写 x , j 写 x , 二者顺序影响最终写入值。

寄存器重命名：

能消除 WAR 和 WAW。

不能消除 RAW。

1.8 八、依赖图

构图规则：

每条指令一个节点。

RAW 画实线边。

WAR 和 WAW 画虚线边。

边方向从先执行的指令指向后执行的指令。

例：

I1: R1 = R2 + R3

I2: R4 = R1 + R5

I3: R2 = R6 + R7

I4: R1 = R8 + R9

依赖：

I1 -> I2 : RAW on R1

I1 -> I3 : WAR on R2

I1 -> I4 : WAW on R1

I2 -> I4 : WAR on R1

重命名后：

把 I3 写 R2 改成写新寄存器 R10。

把 I4 写 R1 改成写新寄存器 R11。

WAR/WAW 依赖被消除，RAW 依赖保留。

1.9 九、列表调度

算法：

1. 建依赖图。
2. 初始 ready list = 入度为 0 的节点。
3. 每个周期选择 ready 中的指令发射。
4. 删除该节点和出边。
5. 新入度为 0 的节点加入 ready。
6. 直到所有节点完成。

优先级:

关键路径更长者优先。
或延迟更大者优先。

考试写法:

给出一个满足依赖图拓扑序的序列即可。
若题目允许寄存器重命名, 先删 WAR/WAW, 再调度。

1.10 十、自测题

题 1: 指令调度能改变程序结果吗?

答案:

不能。调度只能重排不破坏依赖的指令。

题 2: 哪类依赖不能靠寄存器重命名消除?

答案:

True dependence / RAW。

题 3: 冲突图的边表示什么?

答案:

两个变量活跃区间相交, 不能分配同一个寄存器。

题 4: phi 转出时复制插在哪里?

答案:

插在对应前驱基本块的末尾。